

The Hexagonal (Ports & Adapters) Architecture

A Alistair Cockburn apresenta no artigo *Hexagonal Architecture* uma crítica direta à forma tradicional como sistemas são desenhados e pensados, principalmente no modelo em camadas. O texto não é um tutorial passo a passo, nem um guia técnico cheio de exemplos de código. Ele é mais conceitual, quase provocativo, e tenta mudar a forma como o desenvolvedor enxerga a estrutura de uma aplicação.

Logo no início, o autor aponta um problema que você provavelmente já viu na prática: diagramas de arquitetura em camadas são bonitos no papel, mas na vida real não funcionam como deveriam. As “linhas” entre camadas, que deveriam impor limites, acabam sendo ignoradas. O código começa a vazar de um lado para o outro, a lógica de negócio invade a interface, o banco de dados passa a influenciar decisões do domínio, e o sistema vira um acoplado difícil de manter.

A partir disso, Cockburn propõe uma mudança de perspectiva. Em vez de pensar em cima/baixo (UI → aplicação → banco), ele sugere pensar em dentro/fora. Essa é a ideia central do artigo. Existe um “dentro”, que é o núcleo da aplicação (a lógica de negócio), e existe um “fora”, que são todas as coisas externas: banco de dados, interface, APIs, testes, scripts, etc.

Essa mudança parece simples, mas tem uma consequência importante: tudo que está fora deve ser tratado da mesma forma. Não importa se é um banco de dados, uma interface web ou um teste automatizado — todos são apenas formas diferentes de interagir com o sistema. Essa simetria é uma das ideias mais fortes do texto.

Para viabilizar isso, Cockburn introduz os conceitos de *ports* e *adapters*. Um *port* é basicamente uma interface que define uma conversa possível com o sistema. Já o *adapter* é a implementação concreta que conecta o mundo externo a esse port. Por exemplo, você pode ter um port para “buscar usuários” e vários adapters: um para banco de dados, outro para API externa, outro para testes em memória.

O interessante aqui é que o autor não está preocupado com tecnologia. Ele deixa claro que o objetivo é fazer com que a lógica de negócio não dependa de nada externo. Isso significa que, idealmente, você deveria conseguir rodar e testar o core da aplicação sem banco, sem UI, sem framework. Essa independência torna o sistema mais flexível e mais fácil de evoluir ao longo do tempo.

Outro ponto importante que o artigo traz é a ideia de múltiplas portas. Diferente de uma arquitetura em camadas, que geralmente sugere um fluxo linear, aqui você pode ter várias formas de entrada e saída. Pode existir um port para interface web, outro para integração com outros sistemas, outro para testes automatizados, e assim por diante. O desenho em forma de

hexágono não é por causa do número seis, mas porque ele permite visualizar melhor essa multiplicidade de conexões.

O texto também critica a forma como desenvolvedores misturam responsabilidades. Em arquiteturas tradicionais, é comum ver lógica de negócio escondida em controllers, queries complexas no banco contendo regras de domínio, ou até validações importantes presas à interface. Isso cria um acoplamento forte que dificulta mudanças. A arquitetura hexagonal tenta resolver isso isolando completamente o core da aplicação.

Uma consequência prática dessa abordagem é a facilidade de teste. Como o core não depende de infraestrutura, você pode testá-lo diretamente, sem precisar subir banco, servidor ou qualquer coisa externa. Isso melhora muito a confiabilidade do sistema e reduz o custo de manutenção. Além disso, como os adapters são intercambiáveis, trocar uma tecnologia (por exemplo, mudar de PostgreSQL para outro banco) não deveria impactar a lógica de negócio.

Apesar de ser um conceito poderoso, o artigo não entra em detalhes de implementação. Isso pode ser visto como uma limitação. Ele apresenta a ideia, mas deixa muita coisa em aberto: como definir os ports na prática, como organizar o código, como lidar com complexidade real. Isso exige maturidade do desenvolvedor para aplicar o padrão corretamente.

Outro ponto que pode gerar confusão é a própria abstração. Para quem está acostumado com MVC ou arquitetura em camadas, pensar em “ports” e “adapters” pode parecer desnecessariamente complexo no início. Inclusive, há críticas de que o modelo pode esconder diferenças importantes entre tipos de dependência, já que trata tudo externo como equivalente.

Mesmo assim, o artigo cumpre bem o papel de provocar uma mudança de mentalidade. Ele não está tentando ensinar uma receita pronta, mas sim fazer você questionar como organiza seu sistema. A principal mensagem que fica é: a lógica de negócio deve ser independente de tudo. O resto é detalhe.

Se você trazer isso para o contexto real de desenvolvimento, principalmente em frameworks como Rails, dá para ver o impacto direto. A tendência natural do Rails é empurrar lógica para models, controllers e até callbacks, criando dependências implícitas. A arquitetura hexagonal vai na direção oposta: extrair o domínio para objetos independentes (POROs), definir interfaces claras e deixar infraestrutura como detalhe. Isso conversa diretamente com princípios como SOLID e Dependency Inversion.

No fim, o artigo é curto, mas denso. Ele não te ensina “como fazer”, mas deixa claro “o que está errado” nas abordagens comuns e sugere um caminho mais sustentável. É um texto que faz mais sentido quando você já sofreu com código acoplado, testes difíceis e sistemas rígidos. Nesse ponto, a proposta de Cockburn deixa de ser teórica e passa a ser quase óbvia.